University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Graded Lab – Computer Security

CryptoShell Project – Educational Ransomware in Bash

Practical Information:

- **Distribution:** February 16, 2026
- **Deadline:** March 8, 2026 at 11:59 PM on Moodle
- **Instructions: Individual** work. Submit a `.zip` archive on Moodle containing:
  - All Bash scripts (Phases 1–6)
  - The analysis report (Phase 7) in PDF format
- **Grading:** 25 points scaled to /20. Each phase is graded independently.

Ethical and Legal Warning

The techniques presented in this lab are studied in a **strictly academic context** in order to understand the mechanisms used by malicious software and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and subject to criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against automated data processing systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstruction of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by LOPMI Act No. 2023-22 of January 24, 2023.*

Scenario: You are a junior analyst in an incident response team (*CERT*). Your manager asks you to understand how a recent ransomware targeting Linux servers works, then to develop the corresponding detection and recovery tools. To do this, you will **progressively build** an educational ransomware in Bash, called **CryptoShell**, then **switch perspective** to create the countermeasures.

Prerequisites: Chapters 01 (malicious software) and 02 (interpreted viruses), Labs 01 to 04.

Working Method: For each phase, proceed as follows:

- Run `setup_lab.sh` to create a clean test environment.
- Work **only** in the `lab/` directory created by the script.
- Test each phase **before** moving on to the next one.
- Recreate the test environment between phases if necessary.

Provided files:

- `xor_helper.sh` — Helper script containing `generate_key()` and `xor_file()` functions (use with `source`)
- `setup_lab.sh` — Test environment setup script
- `cible1.txt`, `cible2.txt`, `cible3.sh` — Test target files
- `rapport_template.tex` — Analysis report template (Phase 7)

XOR Helper (`xor_helper.sh`) — this script is provided as a black box:

```bash
#!/bin/bash
# CryptoShell XOR Helper
# Usage: source xor_helper.sh

# Generate a random key of N bytes (in hex)
generate_key() {
    local length=${1:-8}
    head -c "$length" /dev/urandom | xxd -p
}

# Encrypt/decrypt a file with an XOR key (in hex)
# Usage: xor_file <input_file> <output_file> <hex_key>
xor_file() {
    local input="$1" output="$2" key="$3"
    local key_len=${#key}
    local i=0
    xxd -p "$input" | tr -d '\n' | fold -w2 | while read byte; do
        local ki=$(( i % (key_len / 2) ))
        local key_byte="${key:$((ki*2)):2}"
        printf "%02x" $(( 16#$byte ^ 16#$key_byte ))
        i=$((i + 1))
    done | xxd -r -p > "$output"
}
```

xor_helper.sh (provided)

Detailed Grading:

Phase	Title	Duration	Points	Difficulty
1	Reconnaissance	30 min	3 pts	⋆⋆
2	XOR Encryption	45 min	4 pts	⋆⋆⋆
3	Propagation & Ransom	45 min	4 pts	⋆⋆⋆
4	Trigger	30 min	3 pts	⋆⋆
5	Stealth	30 min	3 pts	⋆⋆⋆
6	Detection & Recovery	45 min	4 pts	⋆⋆⋆
7	Analysis Report	45 min	4 pts	⋆⋆⋆⋆
	Total	**∼4h30**	**25 pts → /20**	

Exercise 1 (CryptoShell Project — Educational Ransomware).

Phase 1 — Reconnaissance and File Discovery **[3 points]**

Course Reminder

Adleman's Classification and Virus Life Cycle:
Leonard Adleman (1988) proposed a formal classification of malicious programs:

- **Simple programs:** execute their action without reproducing (logic bomb, pure ransomware, spyware).
- **Self-reproducing programs:** copy themselves into other programs or systems (viruses, worms).

The **life cycle** of a virus has 3 phases:

1. **Infection:** the virus copies itself into new hosts (search routine + copy)
2. **Incubation:** the virus is present but inactive (waiting for the trigger)
3. **Disease:** the payload executes (destructive action)

The **file search routine** is the first step of any file-targeting malware. It determines the **scope** of the attack: the more efficient it is (recursive, multi-extension), the greater the impact.

1a) Write a script `recon.sh` that takes a directory as argument and lists all `.txt` and `.dat` files it contains (recursively). For each file, display: the path, size in bytes, and modification date. Use `find` or a recursive loop.

1b) Modify the script so that the total number of files found and the total size are computed. Store the target file list in a file `targets.list`.

1c) **Theoretical question:** In Adleman's classification, where does ransomware fit? Is it a simple or self-reproducing program? Justify.

Phase 2 — XOR Encryption [4 points]

Course Reminder

XOR Encryption and Viral Polymorphism:
XOR (exclusive or) has 4 fundamental algebraic properties:

1. **Commutativity:** $A \oplus B = B \oplus A$
2. **Associativity:** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
3. **Identity element:** $A \oplus 0 = A$
4. **Self-inverse:** $A \oplus A = 0$

Consequence for encryption: If $C = M \oplus K$ (encryption), then $C \oplus K = M \oplus K \oplus K = M \oplus 0 = M$ (decryption). The **same operation** with the **same key** encrypts AND decrypts.

Link with Vernam cipher: XOR with a key as long as the message and truly random constitutes the **one-time pad**, proven unbreakable by Shannon (1949).

Viral polymorphism via XOR encryption: In a polymorphic virus, the viral code is encrypted with a different XOR key at each infection. Only the **decryption routine** (in cleartext) remains constant — it is a **detectable invariant**.

XOR Truth Table:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2a) Explain mathematically why XOR can both encrypt AND decrypt with the same operation and the same key. Give the 4 algebraic properties used.

2b) Manually compute the XOR of the string `"HACK"` with key `0x5A`. Provide the full table (character, ASCII hex, key hex, result hex, result character).

2c) Using `xor_helper.sh`, write a script `encrypt.sh` that: (1) generates a random 8-byte key, (2) encrypts a file given as argument, (3) renames the encrypted file with `.locked` extension, (4) deletes the original, (5) saves the key in a hidden file `.cryptoshell_key`.

2d) Write `decrypt.sh` that: reads the key from `.cryptoshell_key`, decrypts a `.locked` file, restores the original name, and deletes the `.locked` file.

Phase 3 — Propagation and Ransom [4 points]

Course Reminder

Classic Infection Techniques and Anti-reinfection:
Classic viruses use 3 main file infection techniques:

- **Overwriting:** the virus replaces the beginning of the host file (destructive, simple)
- **Appending:** the virus appends itself after the host code (non-destructive)
- **Companion:** the virus creates a file with the same name but higher execution priority

Text diagram before/after appending infection:

```
BEFORE:                      AFTER:
+------------------+         +------------------+
| Host code        |         | Host code        |
| (original)       |         | (original)       |
+------------------+         +------------------+
                              | Viral code       |
                              | (appended)       |
                              +------------------+
```

Anti-reinfection: A virus must avoid reinfecting an already-infected file. Strategies: marker (signature), tail comparison, execution test.

3a) Write `cryptoshell.sh` that: (1) generates a random key, (2) recursively traverses a target directory, (3) encrypts each `.txt` and `.dat` file found, (4) renames to `.locked`, (5) deposits a `RANSOM_NOTE.txt` file in each directory containing encrypted files.

3b) Add an **anti-reinfection** mechanism: the script must NOT re-encrypt an already `.locked` file. Implement this check.

3c) The ransom note must contain: a threatening message, a unique victim identifier (randomly generated), and (fictitious) instructions. Write the `create_ransom_note()` function.

3d) **Theoretical question:** Compare the ransomware infection technique with the 3 classic virus techniques (overwriting, appending, companion). What are the fundamental similarities and differences?

Phase 4 — Trigger (Logic Bomb) [3 points]

Course Reminder

Logic Bombs and Triggers:
A **logic bomb** is a program whose payload only activates when a **condition** is met. Types of triggers:

- **Temporal:** date, time, day of the week
- **Counter-based:** after N executions
- **Event-driven:** file presence, network connection, user login
- **Environmental:** environment variables, machine name

Useful commands:

- `date +%u`: day of the week (1=Monday, . . . , 7=Sunday)
- `date +%d`: day of the month (01–31)
- `date +%m`: month (01–12)

Historical examples: Omega Engineering (1996) — a fired system administrator planted a logic bomb that deleted all production software 20 days after his departure. Dark Seoul (2013) — malware activated at a precise date/time targeting South Korean banks and media.

Link with viruses: A virus with a delayed trigger is a logic bomb mounted on a viral vector. The trigger/payload separation is a **fundamental architectural pattern**.

4a) Modify `cryptoshell.sh` so that encryption only activates if a **temporal condition** is met: the script must check if it is a Sunday (`date +%u` returns 7). Otherwise, the script exits silently.

4b) Implement an **alternative counter-based trigger**: a hidden file `.cs_count` is incremented at each execution. Encryption only activates after 5 executions. After activation, the counter is deleted.

4c) **Theoretical question:** Name two famous historical examples of logic bombs. Explain why antivirus software has difficulty detecting logic bombs before they trigger.

Phase 5 — Stealth and Anti-detection [3 points]

Course Reminder

Stealth Techniques:
A stealthy malware seeks to conceal traces of its activity to delay detection:

- **Timestamp preservation:** modifying a file changes its last modification date (`mtime`). An administrator monitoring dates can detect an anomaly.
- **Temporary file cleanup:** any artifact left by the malware (temporary files, logs) is evidence.
- **Error suppression:** error messages on `stderr` can alert the user.

Key commands:

- `touch -r reference file`: apply timestamps from `reference` to `file`
- `stat -format="%y" file`: display modification date
- `trap "commands" EXIT SIGINT SIGTERM`: execute `commands` on script exit (even on Ctrl+C or kill)
- `2>/dev/null`: redirect `stderr` to `/dev/null`

5a) Add **timestamp preservation**: before encrypting each file, save its modification date, then restore it on the `.locked` file. Use `touch -r` to copy timestamps.

5b) Add a **cleanup mechanism**: use `trap` to guarantee deletion of all temporary files (key, intermediate files) even on interruption (SIGINT, SIGTERM). Redirect stderr with `2>/dev/null` on critical commands.

5c) **Theoretical question:** Line-reordering polymorphism (from Lab 04) and XOR encryption polymorphism (from Lab 02) aim for the same goal: evading signature-based detection. Compare these two approaches: which is more robust? Which leaves a detectable invariant? Could an antivirus detect CryptoShell despite these techniques?

Phase 6 — Detection and Recovery [4 points]

Course Reminder

Indicators of Compromise (IoC) and Detection Types:
An **Indicator of Compromise** (IoC) is an observable artifact that signals an infection or intrusion.

Antivirus detection types:

- **Signature-based** (*pattern matching*): search for known strings or patterns in files (fast, precise, but bypassed by polymorphism)
- **Heuristic**: analysis of suspicious code without known signature — detection of behavioral patterns in source code (e.g., a script that massively encrypts files)
- **Behavioral**: monitoring actions **at runtime** — detecting the program's intent (e.g., mass file access + writing encrypted files)

Useful `grep` commands for analysis:

- `grep -q "pattern" file`: silent mode, exit code 0 if found
- `grep -r "pattern" directory`: recursive search
- `grep -l "pattern" *.sh`: list files containing the pattern
- `grep -c "pattern" file`: count occurrences

6a) Write `scanner.sh` that traverses a directory and detects CryptoShell **Indicators of Compromise (IoC)**:

- Presence of `.locked` files
- Presence of `RANSOM_NOTE.txt`
- Presence of hidden files `.cs_count`, `.cryptoshell_key`
- Suspicious patterns in scripts: `xor`, `locked`, `RANSOM`

The scanner must display a summary report with the number of IoCs found per category.

6b) Write `recovery.sh` that: (1) searches for the `.cryptoshell_key` file, (2) if found, decrypts all `.locked` files in the directory, (3) deletes ransom notes and state files.

6c) **Theoretical question:** Fill in an IoC table for each phase (1–5) with: the technique used, the detectable IoC, the detection type (signature, heuristic, behavioral). Propose a detection strategy combining all 3 approaches.

Phase 7 — Analysis Report [4 points]

A report template (`rapport_template.tex`) is provided. Fill in the following sections:

R1) **Adleman's Classification:** Where does CryptoShell fit in the classification? Simple or self-reproducing program? Justify.

R2) Life Cycle: Draw (in ASCII art or diagram) the complete life cycle of CryptoShell, identifying the 3 classic phases (infection, incubation, disease). Indicate which lab phase corresponds to each step.

R3) Virus, Worm, or Trojan: Is CryptoShell as implemented a virus, worm, or Trojan horse? How would you modify it to become a worm (network propagation)? How would you transform it into a Trojan?

R4) Comparison with Document Viruses: A Word macro-virus and CryptoShell both target data files. What are the fundamental similarities and differences (infection vector, target format, interpreter, persistence)?

R5) IoC Table by Phase: Fill in a complete table [Phase | Technique | IoC | Detection type].

R6) Ethical Reflection: How is the study of malware construction in an academic setting beneficial for cybersecurity? Cite at least one concrete example.

The report is graded on the quality of analysis, not length.